

Learning Microsoft ASP.NET Core Controller Based Web APIs

Inter-Service Communications – Sync-Async Communication

Microservices communication patterns define how independent services exchange data and coordinate with each other in a distributed system. Selecting the right communication pattern is essential for building scalable, resilient, and high-performance microservices applications. These patterns help services communicate efficiently while maintaining loose coupling and fault tolerance.

ASP.NET Core Web API microservices architecture, inter-service communication represents how independent services interact to exchange data. Because each microservice maintains its own domain and database, robust network communication is required to fulfil operations that cross service boundaries.

Types of Microservices Communication

Microservices communication patterns are broadly divided into two categories:

- **Synchronous Communication:** In synchronous communication, one microservice sends a request to another service and waits for an immediate response before continuing execution. It is commonly used for real-time operations where instant results are required.
 - Uses protocols such as REST, gRPC, and GraphQL.
 - Provides immediate response but creates tighter coupling between services.
- **Asynchronous Communication:** In asynchronous communication, microservices exchange messages without waiting for an immediate response, allowing services to operate independently. This approach improves scalability, resilience, and fault tolerance.
 - Uses message brokers such as Kafka, RabbitMQ, and AWS SQS.
 - Enables loose coupling and supports event-driven architectures.

1. Synchronous Communication (Request/Response)

In synchronous communication, one service calls another and **waits for a response** before continuing its execution. This can introduce tight coupling and latency, but it is necessary when a real-time response is required.

- **HTTP/REST APIs:** The most common approach using standard JSON payload delivery. In ASP.NET Core, this is typically implemented using the optimized **IHttpClientFactory** or declarative type-safe HTTP clients like **Refit**.
 - **Best for:** Interacting with external third-party services or quickly connecting legacy internal APIs.
 - **Implementation Example: (AI Generated)**

```
// Program.cs
builder.Services.AddHttpClient<IOrderServiceClient,
OrderServiceClient>(client =>
{
    client.BaseAddress = new Uri("https://order-service-api/");
});
```

```
// Implementation
public class OrderServiceClient : IOrderServiceClient
{
    private readonly HttpClient _httpClient;
    public OrderServiceClient(HttpClient httpClient) =>
        _httpClient = httpClient;

    public async Task<OrderDto> GetOrderAsync(Guid id)
    {
        return await _httpClient.GetFromJsonAsync <OrderDto>
            ($"api/orders/{id}");
    }
}
```

- **gRPC (Google Remote Procedure Call):** A high-performance, contract-first framework that relies on **HTTP/2** and binary serialization (Protocol Buffers).
 - **Best for:** High-throughput, internal server-to-server microservice communication where low latency is critical.
 - **Pros:** Strongly typed contracts defined in language-agnostic .proto files. Native support provided via the Grpc.AspNetCore package.

2. Asynchronous Communication (Event-Driven)

In asynchronous communication, a service sends a message or event to a **message broker** and continues its execution immediately without waiting for a reply. This decouples services, increases fault tolerance, and scales effectively.

- **Common Message Brokers:** RabbitMQ, Kafka, or Azure Service Bus, AWS SQS
- **Abstracting the Broker:** Instead of writing boilerplate infrastructure code, developers use high-level abstraction frameworks like **MassTransit** or **Rebus**.

Example Pub/Sub Flow:

1. **Order Service** creates an order and publishes an OrderCreatedEvent to the broker.
2. **Inventory Service** and **Notification Service** subscribe to that message type and consume it independently to process stock and send emails.

Note : Reference Project: D:\prac\Day37\EmployeeInsuranceDemo\EmpAsyncDemo

For a company-restricted environment where you cannot install RabbitMQ, Docker, Postman, or any other software, you can still practice the concept of synchronous and asynchronous communication between Employee Management and Insurance Management services using only:

- Microsoft Visual Studio Professional
- ASP.NET Core Controller Based Web APIs
- Built-in .NET libraries
- In-memory data
- Swagger/OpenAPI that comes with the Web API project

This is actually a very good training/demo approach because we can understand the architecture first and later replace the in-memory mechanism with RabbitMQ/Azure Service Bus/Kafka in a real project.

Basic Application Demo:

1. Create a New Blank Solution: EmployeeInsuranceDemo
2. Then add two ASP.NET Core Web API projects:
 - a. EmployeeService
 - b. InsuranceService
3. Employee Service
 - a. Models/Employee.cs

```
namespace EmployeeService.Models
{
    public class Employee
    {
        public int Id { get; set; }
        public string Name { get; set; } = string.Empty;
        public string Department { get; set; } = string.Empty;
        public string Email { get; set; } = string.Empty;
    }
}
```

- b. Controllers/EmployeesController.cs
4. Insurance Service
 - a. Models/InsurancePolicy.cs

```
namespace InsuranceService.Models
{
    public class InsurancePolicy
    {
        public int Id { get; set; }
        public int EmployeeId { get; set; }
        public string EmployeeName { get; set; } = string.Empty;
        public string PolicyType { get; set; } = string.Empty;
        public decimal Premium { get; set; }
    }
}
```

- b. Controllers/Insurance Controller.cs

Free, Open-Source, Cross-Platform Framework

5. Set ports for each API for an example:
 - a. EmployeeService → <https://localhost:5001>
 - b. InsuranceService → <https://localhost:5002>
6. As we have done in previous section, demonstrate SYNCHRONOUS communication. This is the easiest concept to teach. Suppose: Employee Service receives a request to create an employee. Employee Service then asks Insurance Service: "Does this employee already have an insurance policy?" The Employee Service waits for the response. That's synchronous communication.
 - a. Add HttpClient to Employee Service
 - b. Create synchronous communication endpoint
 - c. And execute GET /api/employees/1/insurance
7. Here we need to introduce an event. Suppose Employee Service creates an employee. Instead of immediately calling Insurance Service, Employee Service publishes:

EmployeeCreated

Meanwhile the Employee Service doesn't wait.

Employee Service → EmployeeCreated → Queue → Insurance Service

In production the queue might be:

RabbitMQ / Azure Service Bus / Kafka / Amazon SQS

But your restricted company environment doesn't need any of those. We can use the built-in .NET:

Channel<T>

This allows us to simulate a message queue. Let's do it:

8. Create an Event Model: Models/EmployeeCreatedEvent.cs

```
namespace EmpAsyncDemo.Models
{
    public class EmployeeCreatedEvent
    {
        public int EmployeeId { get; set; }

        public string EmployeeName { get; set; } = string.Empty;

        public string Email { get; set; } = string.Empty;
    }
}
```

9. Create an In-Memory Message Queue Interface: Services/IMessageQueue.cs

```
using EmpAsyncDemo.Models;

namespace EmpAsyncDemo.Services
{
    public interface IMessageQueue
    {
        ValueTask PublishAsync(EmployeeCreatedEvent message);
    }
}
```

```

    }
    }
}

```

10. Create an In-Memory Message Queue class: Services/InMemoryMessageQueue.cs

```

using EmpAsyncDemo.Models;
using System.Threading.Channels;

namespace EmpAsyncDemo.Services
{
    public class InMemoryMessageQueue : IMessageQueue
    {
        private readonly Channel<EmployeeCreatedEvent> _channel;

        public InMemoryMessageQueue()
        {
            _channel = Channel.CreateUnbounded<EmployeeCreatedEvent>();
        }

        public async ValueTask PublishAsync(EmployeeCreatedEvent message)
        {
            await _channel.Writer.WriteAsync(message);
        }

        public async ValueTask<EmployeeCreatedEvent> ConsumeAsync(
            CancellationToken cancellationToken)
        {
            return await _channel.Reader.ReadAsync(cancellationToken);
        }
    }
}

```

11. Register the Queue In Program.cs

```
builder.Services.AddSingleton<IMessageQueue, InMemoryMessageQueue>();
```

12. Create Background Worker: Services/EmployeeEventProcessor.cs

```

namespace EmpAsyncDemo.Services
{
    public class EmployeeEventProcessor : BackgroundService
    {
        private readonly IMessageQueue _queue;

        public EmployeeEventProcessor(IMessageQueue queue)
        {
            _queue = queue;
        }

        protected override async Task ExecuteAsync(
            CancellationToken stoppingToken)
        {
            while (!stoppingToken.IsCancellationRequested)
            {
                var message = await _queue.ConsumeAsync(
                    stoppingToken);

                Console.WriteLine(

```

Free, Open-Source, Cross-Platform Framework

```

        $"Processing EmployeeCreated event " +
        $"for Employee: {message.EmployeeId}");

        await Task.Delay(1000, stoppingToken);

        Console.WriteLine(
            $"Event processed for Employee: " +
            $"{message.EmployeeId}");
    }
}
}
}

```

13. Register it in Program.cs

```
builder.Services.AddHostedService<EmployeeEventProcessor>();
```

14. Publish event when employee is created for that Inject(DI) IMessageQueue in controller

15. Then modify POST method with EmployeeCreatedEvent

16. Modify existing controller, below is updated one.

```

using EmpAsyncDemo.Models;
using EmpAsyncDemo.Services;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using System.Net.Http;

namespace EmpAsyncDemo.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class EmpsController : ControllerBase
    {
        private readonly IMessageQueue _queue;
        private readonly IHttpClientFactory _httpClientFactory;
        public EmpsController(IHttpClientFactory httpClientFactory,
            IMessageQueue queue)
        {
            _httpClientFactory = httpClientFactory;
            _queue = queue;
        }

        private static readonly List<Employee> employees = new()
        {
            new Employee
            {
                Id = 1,
                Name = "Ravi",
                Department = "IT",
                Email = "ravi@company.com"
            },
            new Employee
            {
                Id = 2,
                Name = "Priya",
                Department = "HR",
                Email = "priya@company.com"
            }
        };

        [HttpGet]
    }
}

```

Free, Open-Source, Cross-Platform Framework

```
public IActionResult GetEmployees()
{
    return Ok(employees);
}

[HttpGet("{id}")]
public IActionResult GetEmployee(int id)
{
    var employee = employees.FirstOrDefault(x => x.Id == id);

    if (employee == null)
        return NotFound();

    return Ok(employee);
}

//[HttpPost]
//public IActionResult CreateEmployee(Employee employee)
//{
//    //    employee.Id = employees.Count + 1;
//    //    employees.Add(employee);
//    //    return Ok(employee);
//}

[HttpPost]
public async Task<IActionResult> CreateEmployee(Employee employee)
{
    employee.Id = employees.Count + 1;

    employees.Add(employee);

    var employeeCreatedEvent = new EmployeeCreatedEvent
    {
        EmployeeId = employee.Id,
        EmployeeName = employee.Name,
        Email = employee.Email
    };

    await _queue.PublishAsync(employeeCreatedEvent);

    return Ok(new
    {
        Message = "Employee created successfully",
        Employee = employee
    });
}
```

17. Run Employee Service.

POST /api/employees

Request:

```
{
  "name": "Ravi",
  "department": "IT",
  "email": "ravi@company.com"
}
```

Free, Open-Source, Cross-Platform Framework

Employee Service returns:

```
{  
  "message": "Employee created successfully",  
  "employee": {  
    "id": 1,  
    "name": "Ravi",  
    "department": "IT",  
    "email": "ravi@company.com"  
  }  
}
```

But something else happens behind the scenes.

18. You get immediately response with event message, meanwhile observe Visual Studio console also.

```
info: Microsoft.Hosting.Lifetime[0]  
      Content root path: D:\prac\Day37\EmployeeInsuranceDemo\EmpAsyncDemo  
Processing EmployeeCreated event for Employee: 3  
Event processed for Employee: 3
```

NOTE: "Channel<T> is not RabbitMQ."

"For today's restricted-laptop demonstration, we are using .NET Channel<T> as an in-memory message queue. In production, this abstraction can be backed by RabbitMQ, Azure Service Bus, Kafka, etc."

❧ ❧ ❧ leave No Stone Unturned ❧ ❧ ❧